

**Extracting Structured Financial Data from PDF Bank Statements Using a Vision Language
Model**

DS677 - Final Project Report

Sivaragha Buddana & Tara Walenczyk

Video Presentation Link: <https://canva.link/p1stjgsftmo5irh>

Link to code/repo: <https://github.com/NJIT-DS677-Deep-Learning/bank-statement-converter-final-project>

Overview

In wealth management and financial technology, one of the most persistent data challenges is sharing, transferring, and utilizing data that can only be extracted in the form of a PDF. This format is not structured or tabular, which leads to either manual extraction or PDF parsing systems that transform the data, both of which are error-prone, require specific builds for a particular PDF structure, and take a long time.

This project fine-tunes a vision-language model, specifically Donut (Document Understanding Transformer), which pairs a Swin Transformer vision encoder with a BART-style decoder to jointly read layout and text without a separate OCR step to extract, categorize, and structure transaction data from common bank statement formats. The model takes PDF pages as input and outputs structured data containing transaction date, description, debit/credit amount, running balance, and account-level metadata.

Traditional OCR approaches extract raw text but do not preserve page layout context. They can read a bank statement but have no understanding of which number is a transaction value and which is a date. Vision-language models address this by learning about the visual layout and textual content simultaneously. Fine-tuning an open-source model across multiple institution formats should, in principle, allow generalization across structural differences without requiring hand-built extraction rules per institution.

In the industry practice of supporting wealth managers with historical data transformation, the PDF format becomes a blocker for many clients. The historical records are the largest issue and a reliable extractor would reduce manual work and reconciliation substantially. Beyond wealth management, the same approach applies to expense reconciliation for small businesses, automated bookkeeping, and other financial document types such as brokerage statements and tax documents

1. Architecture

1.1 Donut

Donut is an OCR-free document understanding model. Rather than the conventional two-stage pipeline (run OCR, then interpret the extracted text), Donut maps a document image directly to a structured text output using a single end-to-end trained encoder-decoder network.

Through the Vision encoder (Swin Transformer), the input document image is divided into patches and processed by a hierarchical Swin Transformer. Swin computes self-attention within local windows and progressively merges patches across stages, which lets it capture both fine detail (individual characters) and broader layout structure (this block is a table column; that block is a header) without the quadratic cost of global attention over every pixel patch. The encoder outputs a sequence of visual embeddings of shape (batch_size, seq_len, hidden_size).

Through the Text decoder (BART-style), output tokens are generated autoregressively, one token at a time, each conditioned on all previously generated tokens. Critically, at every generation

step the decoder cross-attends to the full set of encoder visual embeddings. This is not a one-time handoff of image information; the decoder continuously re-queries the image representation as it builds its output.

Output format. Donut's output is a flat token sequence that decodes into a structured string. Structure is imposed by special tokens that mark field boundaries, which are added to the tokenizer vocabulary during fine-tuning.

1.2 Fine-tuning setup

We defined a schema-specific token vocabulary for bank statements. Field tokens (each used as an open/close pair, e.g., `<s_date>...</s_date>`) were:

- Statement-level: `s_account_holder`, `s_account_number`, `s_bank_name`, `s_statement_period_start`, `s_statement_period_end`, `s_opening_balance`, `s_closing_balance`
- Transaction-level: `s_transactions`, `s_transaction`, `s_date`, `s_description`, `s_debit`, `s_credit`, `s_balance`

Plus structural tokens `<s_donut-statement>` / `</s_donut-statement>` marking document boundaries.

A true_label statement record is serialized into a target token sequence such as:

```
<s_donut-statement><s_account_holder>Laura Beck</s_account_holder>
<s_account_number>...5506</s_account_number><s_bank_name>Chase
Bank</s_bank_name> ...<s_transactions><s_transaction><s_date>2026-05-12</s_date>
<s_description>STARBUCKS #1689</s_description><s_debit>66.91</s_debit>
<s_credit></s_credit><s_balance>3433.51</s_balance></s_transaction>... </s_transactions></s_donut-
statement>
```

We used `model.decoder.resize_token_embeddings(len(tokenizer))` to grow the embedding size with the added tokens. Additionally, `model.config.decoder_start_token_id` is set to `<s_donut-statement>` so that generation always begins at the same document context.

Training Configuration

Parameter	Value
Base Checkpoint	naver-clova-ix/donut-base
Image Resolution	960 x 1280

Max Sequence Length	768 Tokens
Batch Size	1
Optimizer	AdamW
Learning Rate	3e-5
Epochs	15
Hardware	NVIDIA A100 (10GB MIG slice), NJIT Wulver HPC
Training Time	~26 minutes

We used a plain PyTorch training loop with teacher forcing for transparency and easier debugging. Checkpoints were saved only when validation loss improved, which functions as implicit early stopping.

2. Technical Innovations / Comparison to Related Work

2.1 Versus traditional OCR pipelines

Conventional extraction runs OCR (e.g., Tesseract, PaddleOCR) to recover text and bounding boxes, then applies rules or a downstream model to assign semantic meaning. Two structural weaknesses occur through this method:

- Error propagation, where OCR mistakes are unrecoverable downstream; the interpretation stage never sees the original pixels.
- Layout blindness: raw OCR output is a bag of strings with coordinates; distinguishing a transaction amount from a running balance from a date requires either hand-written positional heuristics per statement format, or a separate layout model.

Donut's central innovation is eliminating the OCR stage entirely. Because the encoder and decoder are trained jointly on the image-to-structured-text objective, layout reasoning and content reading are learned together rather than pieced together later on.

2.2 Versus OCR + layout-aware transformers (LayoutLM family)

LayoutLM, LayoutLMv2/v3, and similar models improve on naive OCR pipelines by ingesting OCR text plus bounding box coordinates plus (in later versions) image patches, then applying a transformer over that joint representation. They perform well, but still depend on an upstream OCR engine, inheriting its errors, its latency, and its per-language/per-font limitations.

Donut removes the dependency on upstream OCR; however, using donut character recognition resides in the same weights being fine-tuned rather than in a separate, independently pretrained OCR component. As our results show, this demands substantially more training data than a pipeline that gets any recognition "for free" from a mature OCR engine.

2.3 Versus general-purpose vision-language models

Large general-purpose VLMs (GPT-4V, Qwen2-VL, PaddleOCR-VL) can be prompted zero-shot to extract statement fields. They benefit from enormous pretraining and require no task-specific training data. Their disadvantages for this use case are cost per document, latency, data-residency concerns for financial records, and no guarantee of schema-conformant output.

The comparison we intended to run, fine-tuned specialist versus zero-shot generalist, is discussed as future work in Section 5, but the fine-tuned model's performance (Section 4) did not reach a level where the comparison would have been informative.

3. Self-Contained Demo

3.1 Process

Steps:

1. PDF pages are converted to images at 150 DPI
2. Each page image is processed by DonutProcessor and passed to `model.generate()`, producing a token sequence
3. The sequence is parsed back to structured records via `token_sequence_to_json()`, using tolerant regex matching since raw generations are frequently malformed
4. Statement-level fields are taken from page one and transaction lists are concatenated across pages
5. Output is printed as a table and written to CSV

The demo ([src/demo.py](#)) implements the full path from the raw PDF to structured output.

3.2 Usage

Example run statement:

```
python src/demo.py --checkpoint ./donut-bankstatement --pdf statement.pdf
```

3.3 Example Input and Output

Example Input

Charles Schwab

Brokerage Account Statement

Account Holder	Michael Wood
Account Number	...2682
Statement Period	2025-10-24 – 2025-11-23
Opening Balance	\$1787.67
Closing Balance	\$-1177.48

Activity

Trade Date	Description	Debit	Credit	Balance
2025-10-27	TARGET STORE #7419	135.30		1652.37
2025-10-27	DIRECT DEP PAYROLL	679.69		972.68
2025-11-01	STARBUCKS #686		411.16	1383.84
2025-11-01	UBER TRIP 9404	381.87		1001.97
2025-11-02	TARGET STORE #4565	418.45		583.52
2025-11-06	INTEREST PAYMENT	189.10		394.42
2025-11-08	CHECK #5365	441.93		-47.51
2025-11-14	COMCAST CABLE	529.85		-577.36
2025-11-17	ATM WITHDRAWAL #8665	600.12		-1177.48

Model Output

account_holder: Katha Ne
account_number: None
bank_name: se<s_bank_name>se
statement_period_start: None
Transactions found: 0

Raw Generated Sequence

<s_donut-statement><s_account_holder> Karin Nelson<s_account_number>550
</s_account_number> Cha Bank<s_statement_period_start>2606<s_statement_period_end>
2606...<s_transactions><s_date>2606<s_description>GETORE26</s_description>
11399<s_balance>3882</s_transaction><s_date>2607<s_description>ELLIL38...

The results of this example are explained in sections 4.3 and 4.4.

4. Application to Datasets

4.1 Dataset construction

There are no, or very minimal, public datasets that exist, so we created synthetic bank statements. Individuals working on this problem report building private, internally annotated corpora for the same reason.

We therefore built a synthetic statement generator, the same methodological approach the original Donut authors took with SynthDoG. The generator produces names via Faker and randomizes account numbers and statement periods. Transaction descriptions are drawn from twenty templates written in the abbreviated style banks use (AMAZON MKTPLACE PMTS, STARBUCKS #1689, ATM WITHDRAWAL #4471). Balances are computed sequentially across transactions, so each statement is consistent. Statements are then moved to PDF format with ReportLab in two visual layouts, a Chase-style layout with a colored header bar and gridded table, and a Schwab-style layout with a boxed summary and gridless table. The PDF's are then transformed to images at 150 DPI.

Dataset size: 200 statements total - 140 train / 30 validation / 30 test, each containing 8-20 transactions.

4.2 Training Trials

Run	Resolution	Epochs	Final Val Loss	Notes
1	640 x 896	4	1.6563	Reduced resolution after CUDA OOM at batch size 2
2	800 x 1088	15	.2906	Overfitting visible after ~epoch 11
3	960 x 1280	15	.2030	Best validation loss

4.3 Evaluation Results

We evaluated on all 30 test statements using two metric families, reported together:

- **Exact match** - a field is correct only if the predicted string matches the true label character-for-character. This is the operationally meaningful bar for financial data.
- **Character similarity (fuzzy)** - difflib.SequenceMatcher ratio between predicted and true strings, 0.0-1.0. This reveals how close the model got, which exact match alone hides.

Statement level results (n = 30):

Field	Exact match	Char similarity
account_holder	0.00	0.452
account_number	0.00	0.249
bank_name	0.00	0.246
statement_period_start	0.00	0.168
statement_period_end	0.00	0.227
opening_balance	0.00	0.016
closing_balance	0.00	0.006
Mean	0.00	0.195

Transaction Results (n=30):

Metric	Exact	Fuzzy
Precision	0.00	0.00
Recall	0.00	0.00
F1	0.00	0.00

Across 30 test statements containing 400 true label transactions, the model produced 26 parseable transaction records and matched zero of them under either criterion. 24 of the 30 statements produced no parseable transactions at all.

4.4 Error Analysis

The results are unambiguous, and the raw generation output explains what occurred during training. The model learned the schema structure but not character-level content.

Generated sequences contain the correct special tokens, in approximately the correct order, with a repeating transaction block pattern. The model clearly understood "a statement consists of header fields followed by a repeated list of date/description/debit/credit/balance groups". However, field values are systematically malformed. Dates render as 2606 rather than 2026-06-24. Merchant names render as GETORE26, WOLEOODAST47, STAUC #39, which are recognizably derived from real merchant strings (TRADER JOES, WHOLE FOODS, STARBUCKS) but are character-corrupted.

Some of the factors that may be impacting:

1. **Training data scale** -140 examples is roughly three to four orders of magnitude below what Donut's authors used. Structural patterns repeat identically in every example and are learned quickly; specific character sequences differ in every example, providing far less signal per token.
2. **Exposure bias** -Validation loss is measured with teacher forcing; at each step, the model is fed the correct previous token. At inference, it is fed its own previous prediction. Once one character is wrong, every subsequent token is conditioned on corrupted context, and the sequence drifts. This is why a respectable validation loss of 0.20 coexists with near-total free-running generation failure, and it is the single most important caveat in our results.
3. **Precision requirements of the task** - Our transaction matching requires date, debit, credit, and balance to all match. Even a model getting most characters right would score near zero, which is appropriate for financial reconciliation, where a wrong digit is a wrong record, but means the metric offers no partial credit for near misses.
4. **Limited Diversity** - The artificial data consisted of just two bank statement templates. This limitation can have led the neural network to memorize rather than learn the structure of these templates. In practice, there can be variations of different fonts, spacing, column placements, headers, transaction table arrangements, etc., in the actual bank statements. Therefore, it will be hard for a model trained on a few templates to cope with such variations. That said, performance was nearly identical across both templates in our test set; the Chase-style statements yielded 14 parseable transactions from 203, and the Schwab style yielded 12 from 197, which suggests layout complexity was not the differentiating factor in this particular failure, but instead the data scale was the binding constraint.
5. **Long output sequences** - The length of each statement is between 8 and 20 transactions, so a lot of tokens and fields need to be generated in the output sequence by the model. With long

sequences, there is a risk of having more errors. An error at the beginning of the output will impact the rest of the output transactions.

5. Future Applications

5.1 Immediate next steps for this model

- **Scale the synthetic dataset by 10–100×** - Generation is fully automated and costs only compute, so 10,000-100,000 statements is entirely feasible and is the change most likely to move results.
- **Staged training** - Train first on statements with few transactions (short sequences, less error compounding), then extend to full-length statements.
- **Beam search or constrained decoding at inference** - Greedy decoding compounds errors, and constrained decoding that enforces the schema grammar (a date field must contain a valid date pattern) would prevent the malformed-token failures we observed.
- **The zero-shot VLM comparison** - Benchmarking Qwen2-VL or PaddleOCR-VL prompted zero-shot on the same test set would establish whether a general-purpose model already solves this, and would contextualize the fine-tuning result either way.
- **Introduce more variation in the layout** - There is very little variability in the layout because only a few statement templates have been used to prepare the dataset. More diversity can be introduced by using more statements from a variety of banks, layouts, fonts, spaces, logos, columns, and transactions.
- **Additional post-processing and validation steps** - The output generated could be validated after being decoded using some basic validation steps. The dates could be validated to ensure they are in a proper format, balances could be validated to ensure proper formatting in currency, and transaction fields could be validated for any missing or duplicate entries. One future addition could be to include an arithmetic self-evaluation for the Bank statements that satisfy a bank statement constraint: opening balance plus net transactions equals closing balance. You can check extracted output against that with no true labels needed. This would add a quality signal at inference and can enhance confidence in the model. For example, in one of our test runs, the model emitted `<s_opening_balance>6049<s_opening_balance>1649`, which showed two different opening balances. The addition of a check on the actual sum of transactions to the ending balance could give information to how well the model is performing and could even be potentially used to train the model.
- **Scheduled sampling** - Scheduled sampling may solve the discrepancy between training and testing. In the case of scheduled sampling, when we are training our model, the decoder does not always get the true previous token; rather, it gets its own prediction at some points in time.

5.2 Future Novel Applications

One future application could be confidence-based routing where, rather than aiming for perfect extraction, token-level generation probabilities are attached and any low confidence values are routed for human review. In wealth management operations, this can be arguably more valuable than raw accuracy since the user will need to know which records they need to check, and which ones the model is very confident on.

Another future application could be taking the pdf statement parsing a step further to also use phone photographs of statements rather than clean digital PDFs. There can be a lot of cases where the PDF

statements are in a physical copy format, and creating the model in a way to accept and parse images of statements would be extremely powerful.

6. Conclusion

This project investigated the possibility of using a fine-tuned Donut model to extract the structured data of financial documents contained within PDF bank statements without the need for an OCR pre-processing step. The model was successful in learning the overall structure of the desired schema, including the sequence of account-related fields and the repeated transaction blocks. However, it failed to achieve good character-level extraction of dates, merchants' descriptions, account details, and numerical amounts. In the end, the model obtained a mean exact-match score of 0.00 and a mean character similarity score of 0.195, but no definite matches were found for any of the 400 transactions in the true labels.

The problem is not with the architecture as a whole, but rather with the limited size and diversity of training data. Using just 140 training samples, the model managed to identify repetitive structure tokens much more easily than individual character sequences that make up the fields. Exposure bias, the length of output sequences, and the precision needed to operate on financial data were additional factors affecting the gap between validation loss and real-world inference capabilities. Yet, the fact that the model managed to recreate the basic structure gives grounds to believe in its promise.

Further improvements could include expanding the scope of the artificial dataset, adding more layout variation, employing staged training, and using beam search/constrained decoding. It would be useful to compare the performance of the fine-tuned model with that of zero-shot vision-language models and traditional OCR-based systems to determine the best solution. In summary, this project highlights both the potential and the current challenges associated with OCR-less document comprehension in financial data extraction. Although this model cannot be used in production at this point, the pipeline developed in this project will provide a good starting point for experiments in bank statement processing and accounting.

7. References

Kim, G., Hong, T., Yim, M., Nam, J., Park, J., Yim, J., Hwang, W., Yun, S., Han, D., & Park, S. (2022). *OCR-free Document Understanding Transformer*. ECCV 2022. arXiv:2111.15664

Hugging Face model card: naver-clova-ix/donut-base. <https://huggingface.co/naver-clova-ix/donut-base>

Huang, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2022). LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. Retrieved from <https://www.microsoft.com/en-us/research/publication/layoutlmv3-pre-training-for-document-ai-with-unified-text-and-image-masking/>

8. Appendix

Repository Structure

```
bank-statement-converter-final-project/  
src/  
  schema.py          # field tokens, JSON <-> token sequence conversion  
  generate_statement.py # synthetic statement generator, 2 templates  
  build_dataset.py    # train/val/test split construction  
  finetune_donut.py   # fine-tuning loop  
  demo.py            # PDF -> structured transactions  
  eval.py            # exact + fuzzy evaluation metrics  
  eval_on_testset.py  # full test-set evaluation driver  
  finetune_job.slurm   # SLURM batch script (NJIT Wulver, course_gpu partition)  
  logs/              # training logs from all runs  
  eval_results_test.json # final evaluation output  
  examples/          # sample rendered statements  
  requirements.txt
```

Run Reproduction

```
pip install -r requirements.txt  
python src/build_dataset.py --n 200 --out ./dataset  
sbatch finetune_job.slurm  
python src/eval_on_testset.py --checkpoint ./donut-bankstatement --dataset ./dataset
```